

Eksperyment 4.4 – Retrieval-Augmented Warm-Start Decoding

Czy baza wektorowa (Qdrant) z bibliotekę "sygnatur zachowania" wcześniej poznanych sygnałów CAN może dać modelowi LLM lepszy punkt startowy do dekodowania NOWEJ, nieznanej ramki – zamiast klasycznego zgadywania od zera ("Cold Start")?

1. SKĄD TEN POMYSŁ – KONTYNUACJA EKSPERYMENTU 4.1

PUNKT WYJŚCIA

Eksperyment 4.1 pokazał, że modele LLM (Claude, GPT, DeepSeek, Gemini) w trybie "Cold Start" (zero-shot, bez żadnej wcześniejszej wiedzy o pojeździe) mają wyraźny, powtarzalny problem: **bajty pakujące kilka niezależnych flag bitowych** (np. światła, drzwi, kierunkowskazy w jednym bajcie) są prawie zawsze błędnie interpretowane jako pojedyncza wartość skalarna. Jedynym skutecznym lekarstwem znalezionym wtedy był **deterministyczny "hybrydowy override"** – twarda, programowa korekta odpowiedzi LLM na podstawie analizy skoków wartości w czasie (0% → 90-97% detekcji flag).

Pytanie badawcze Eksperymentu 4.4: czy zamiast twardej korekty PO fakcie, da się dać modelowi lepszą podpowiedź PRZED odpowiedzią – na bazie podobieństwa do sygnałów już wcześniej poprawnie rozpoznanych (przez inny mechanizm: wektorowe wyszukiwanie podobieństwa)?

2. ARCHITEKTURA – DWIE FAZY, DWA RÓŻNE ZBIORY DANYCH

FAZA A – BIBLIOTEKA (OFFLINE, BEZ SPRZĘTU)

Budowa indeksu Qdrant

1. Syntetyczny generator
`generate_traffic_diverse.py` tworzy **40 różnorodnych konfiguracji** CAN ID (5 wzorców: sygnały ciągłe, flagi bitowe, flagi rozproszone, bajty mieszane, mieszanki) – `seed=42`.
2. Dla każdego z 152 sygnałów: cechy behawioralne (rozkład wartości, wielkość skoków, oscylacja, "dwell fraction") – bez użycia sieci neuronowej, ręcznie zaprojektowany wektor 7-wymiarowy.
3. Wektory + metadane (typ sygnału, skala, offset) trafiają do **Qdrant** (tryb lokalny/embedded, bez serwera).

FAZA B – TEST NA ŻYWO (PRAWDZIWY SPRZĘT)

Przechwytywanie i ewaluacja

1. PEAK PCAN-USB generuje NOWY ruch – **inny seed=999**, 8 CAN ID, fizyczna magistrala CAN, ESP32+MCP2515 (firmware `esp_experiment_1_1.ino`, niezmienione) sniffuje i przesyła przez WebSocket.
2. 100 realnych okien "Cold Start" przechwyconych z żywej magistrali.
3. Dla każdej próby: zapytanie do Qdrant per bajt → odpowiedź tekstowa dopisana do promptu → **parowane** zapytanie do 4 modeli LLM (baseline bez odpowiedzi, warmstart z odpowiedzią, na TYCH SAMYCH danych).

3. KLUCZOWE DECYZJE METODOLOGICZNE (TO JEST TO, CO BRONI PRACĘ)

01

Różne ziarno losowości (A≠B)

Biblioteka Qdrant (seed=42) i żywy ruch testowy (seed=999) pochodzą z innych konfiguracji. Test mierzy prawdziwą **generalizację** do podobnych-ale-nieznanych sygnałów, nie trywialne odnalezienie identycznego wpisu.

02

Porównanie parowane

Każdy model widzi **DOKŁADNIE** te same 100 prób w obu warunkach (baseline/warmstart) – silniejsze statystycznie niż niezależne próby losowe dla każdego wariantu.

03

Prawdziwy sprzęt, nie tylko symulacja

Dane testowe pochodzą z fizycznej magistrali CAN (PEAK PCAN-USB + ESP32/MCP2515), zweryfikowanej dwukierunkowo (heartbeat + echo) przed zbieraniem danych.

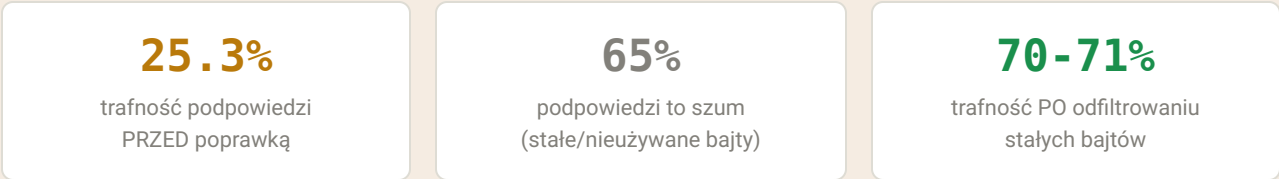
04

Cechy ręczne, nie "czarna skrzynka"

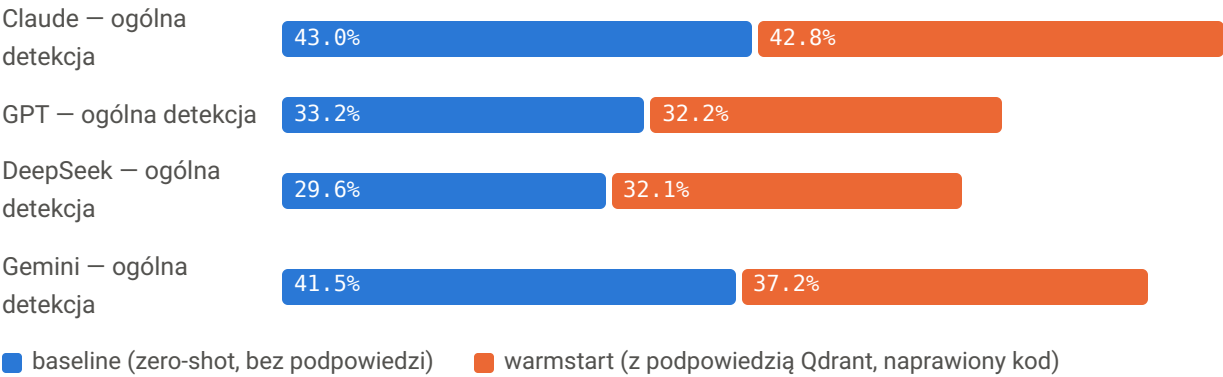
Wektor cech (7 wymiarów: rozkład, skoki, oscylacja) jest w pełni jawny i wytłumaczalny – Qdrant nie wymaga sieci neuronowej do generowania embeddingów.

ZNALEZIONY I NAPRAWIONY BŁĄD – ELEMENT RZETELNOŚCI METODYCZNEJ

W trakcie analizy zauważono, że podpowiedzi Qdrant były generowane dla **każdego** bajtu ramki, w tym bajtów bez żadnego rzeczywistego sygnału (padding). Stały bajt (bez zmian w całym oknie obserwacji) statystycznie przypomina "zawsze wyłączoną flagę" i dostawał fałszywie pewną (score≈1.0) podpowiedź – czysty szum myślący model.



4. WYNIKI FINALNE – 4 MODELE, N=100 SPAROWANYCH PRÓB, 0 BŁĘDÓW



KATEGORIA (AGREGAT 4 MODELI)	BASELINE	WARMSTART	Δ
scalar (n=24)	70.3%	73.3%	+3.0pp
bit_flag (n=116)	17.1%	13.5%	-3.5pp
partial_scalar (n=4, mała próba)	87.5%	100.0%	+12.5pp*

Δ BIT_FLAG PER MODEL	CLAUDE	GPT	DEEPSEEK	GEMINI
zmiana	-4.2pp	-3.7pp	-1.1pp	-5.2pp

* n=4 sygnały łącznie (pojedynczy typ w żywym korpusie) – nie uogólniać. Wzorec pogorszenia na bit_flag jest SPÓJNY U WSZYSTKICH 4 MODELI – to nie przypadek jednego modelu.

5. WNIOSKI KOŃCOWE I OGRANICZENIA

- **Efekt ogólny jest bliski zeru** (od -4.3pp do +2.4pp zależnie od modelu) – to NIE jest darmowe usprawnienie, wbrew wstępnemu (zazumionemu) wynikowi sprzed poprawki błędu.
- **Flagi bitowe konsekwentnie tracą u WSZYSTKICH 4 modeli** (-1.1 do -5.2pp) – dokładnie tam, gdzie odpowiedź miała najbardziej pomóc. Miękką sugestia nie zastępuje twardej, deterministycznej korekty z Eksperymentu 4.1.
- **Prawdopodobny mechanizm:** odpowiedź Qdrant ma tylko ~70% trafności na nieznanych danych – błędna odpowiedź nie myli modelu losowo, tylko *potwierdza jego już istniejący błąd* (Claude i inne modele domyślnie traktują pakowany bajt jako skalar).
- Sygnały ciągłe (scalar) generalnie korzystają (+3.0pp) – tam błędna odpowiedź rzadziej trafia w silny, istniejący bias modelu.

OGRANICZENIA DO JAWNEGO ZAZNACZENIA

- Cechy ręczne (nie neuronowe) – prostsze, wytłumaczalne, ale mogą tracić informację względem uczonych embeddingów.
- Mały rozmiar próby dla `partial_scalar` (n=4) w żywym korpusie.
- Nierówny rozkład prób między CAN ID (artefakt wyzwalacza opartego na czasie).
- Próg podobieństwa (0.85) nie był strojony/optymalizowany.